

#oop:-

1. حاجت هبیت هبر عینا ب class

• class

↳ function - attribute

↳ اگر رو class یه دونه ش سرفا کنه
فیصل

↳ class - attribute - function

• object

↳ به کمکش داده استن class

class object = new class ();

ex: Student student = new Student ();

↳ constructor
access for attribute & function
student. name of attribute
student. name of functions ()

• Access modifiers → attribute & function

• private → هبیت مدقیق، الله الاموال class
student. id
که هبیت هبر یه

• public → بقدر الله من ای مکانه برا class
ار برا

• protect → من ای مکانه بقدر
لتر عینا

• "default" → مت دکت حاجت
منالکراس دول

↳ بقدر الله الله جوار برا class
لتر مت بقدر الله برا package لتر جوار package لتر، لتر لکرا

• Constructor

هو الباب الذي يفتح نفسه
class

• عبارة اعلية جزءه يخص
منه كلاس

• اسم نفس اسم كلاس ← فيه function وليس function
بسرعة return

student class

→ default constructor

• لم يكتسبت كلاس ينشأ automatic دى ان ماضى

• ممكن تعمل اكثر من constructor ياها وخص من default اوالا
ياخر مفضل

ex- student class (const const)

}

}

• يستخدم all methods مع constructor كلاس

• Encapsulation " التغليف "

• هدفك كل attribute بحيث مش
data او ايضا ماعطاه غير

⇒ بقدر اعل attribute
على الكلاس دى

private ← attribute

set/get ← function

public ←

ex- setId, getId

• this keyword → ليعرف اسم attribute نفس الاسم

parameter بياع function زى كلاس لو جوا كلاس

id = id → id بياع

function

object from this
this.id = id

parameter كلاس كلاس

Inheritance

يستخدمه في إنشاء اقل التكرار واعنه من maintenance او تحسين "مبادئ الكود"

بعد ذلك و باضا اشترك في Parent و عرود للابن

في public class Student extends BaseEntity

لوانا عامل attribute ← private عوا الاب من ههنا ههنا
باعتبار لازم استنرم ههنا / ههنا

صيفنص يبق ههنا يورات من آتر من ههنا يورات من
وام فقط

لوانا ههنا function بنفس الاسم للاب ههنا الاب

Life Cycle of Inheritance

لا يغيرت من الاب من انا ههنا ههنا ههنا ههنا
ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا
ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا

لأطب لو ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا
ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا
ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا

ex- public Student() {
 super("default");
}

ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا
ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا

لوانا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا
ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا ههنا

لنختم به `protected` لازم يكتب اول `Construct`
 . الفرق بين `protected` ، `default` و `inter` :
 لو اننا في الـ `class` نكتب `protected` واما `default` و `inter` في
 نكتب `function` من الـ `class` لكن ايه ما نفس `package` و ايه ما
`package` تاني

لما في `function` لو بيت `class` المستفدين
 . فانه `package` ← هيسترون الاين .
 . برا `package` ← هيسترون `protected` بس .

الخلاصة لو بتعمل `inherit` ← سوار ما نفس `package` او برا
 في المستفدين `package` عايز تقدر تكتب .
 ايه لو بتعمل `package` عايز `main` من مستفدين `package`
 برا `package` .
package غير inherit

لو عندي `function` بنفسي الـ `parent` و `child` ايه
 الـ `child` هيحصل

لما بيحصل `override` من `child` على `parent`
 فهنفذ الـ `child` .

→ `override`

بغير تنقيح نفن `function` من `child` .

طب لو عملت `override` و عايزه ينفذ عايد الـ `parent` او ينفذ

الـ `parent` هو الـ `super` ← هستخدم `super`
 لما قبله اروح لـ `parent`

ex-

```

public void print() {
    super.print();
}

```

 ex-

`super`
 refers to
parent

يستخدم notation `@Override` فوق function للمعاملات
لما interface
يصف لبرامج كيف الـ interface
له نفس المظهر مع الـ interface

لما لو اننا عايز اعد function من الـ interface
مبقتش بعد الـ interface
keyword `final` `void` `print` `of` `int` `x`
بمختارها مع الـ interface والـ interface

ex- `final int x = 5`
`public final void print of int x` Con of override
لو عندنا class من عايز نيرثه هستخدم معناه
ex- `public final class B {`
معناه اننا اعد الـ object ولكن مبدئية
بقدر نيرثه.

لو عندنا دالة من عايز اعد الـ implementation
الـ header
ex- `public abstract void print (-, -) {`
حاجه ملاحظة معناه
header `function` `abstract`

دالة تعمل
ex- `public abstract class B {`
لما لو اننا عايز نيرثه
ex- `abstract method`
لما لو اننا عايز نيرثه
ex- `abstract class`
لما لو اننا عايز نيرثه

* polymorphism

تعدد الاوصاف

نفس fraction ولكن لا كذا شكل او يطبقها
بالتر من صورة م عن طريقه كل overiding

← صانع فضاء ← obj → من abstract class ← طريقة مميزة

4. $\text{BelEntropy}(\text{belEntropy}) = \text{New BelEntropy}() \quad \square$

@ override

```
@Override  
public void printAllData() {
```

3 3

this way called Anonymous class

← لو ان صيغتين من رواد كل من `function` والرمز `abstract` فبذل فعل
 رواد `abstract` ← اتوج الرمز `Interface`

* Interfale

factory JS → abstract class
 abstract → body

• public sector fraction v_1 .

ex. -

public interface Bldg {

```
public abstract void print1(); ~ ~ ~ void print4();
```

$\sim \sim \sim \sim \sim \rightarrow \text{word matrix}$

public, abstract. $\leq$ نقل

publ. c ye b default

extend import by using interface

abstract v object
class

۴۰ حلقه نقل 4×4 به 4×4 به طریق تائید

Ex: parent divorce = new child C

Base entity ~~base entity~~ = real student (1);

Base entity & entity = new student (C)
 ↳ reference type → parent ↳ object type "child" (4)

ممكن بعد ذلك ان يكون entity عبارة عن teacher لكن
 entity يجب ان يكون child لـ entity.
 ex: entity = new Teacher (C);

↓
 رد: يصير من مظاهر polymorphism.

الفرق بين العنيتين اللتين في الـ polymorphism
 لغير اولى على الـ child لكن العكس يعمل لان
 الاب يس.

↳ static

او لم يحدد قبل
 static
 بربطه بـ class

فيكون static
 static على مستوى
 كل برنامج

* يتصل مع الـ static class
 واما ان كان على مبنى class جديد او
 فيسجل يحتفظ بالبيانات وليس في الـ static
 الـ class.

ex: public static Integer
 counter = 0;

فيكون static
 فيكون static
 ex: Employee Counter
 static

ما يفعله عن
 على مستوى class
 من الـ static

ما فعله الـ static مرتبط بـ class من
 static

↳ static → function

يتصلن امة الـ static

على مستوى class
 فيكون static
 فيكون static
 static

↳ class can implements many interfaces
↓
دو سے extends کر سکتا ہے۔

لوگیکل فکریہ function کا اکثر interface
معمولاً override کر دیا جاتا ہے۔

← interface سے implement کرنے والے class
"implements" interface

← interface سے extend کرنے والے class
"extends" interface

→ Java prevent multiextend for class, allow interface

← function جو interface میں لکھا ہوا ہو
body دینے کی بجائے default کی

ex: default void get() {
 sort
}

وہ override کر سکتا ہے

← static function جو interface میں لکھا ہوا ہو
وہ static ہی رہتا ہے، override نہیں کر سکتا
اس لیے interface میں static function لکھنا
معمولاً نہیں ہوتا

ex: static void edit() {
 //
}

میں override کر سکتا ہے

← variable جو interface میں لکھا ہوا ہو
وہ final و static ہوتا ہے

ex: int id = 5

اس کا override نہیں کر سکتا
Interface